Обучение нейронной сети

В рамках вводного курса мы начали изучать нейросети. Кроме того, нам знаком важный элемент алгоритма нейронной сети — умножение матриц.

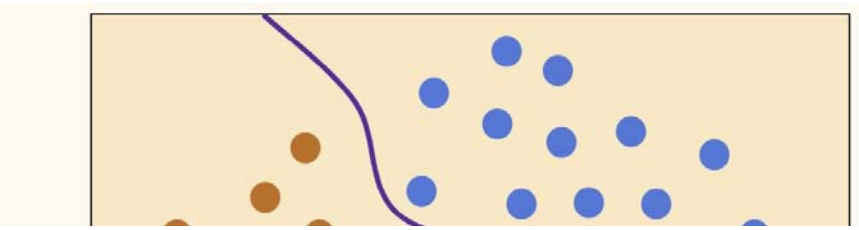
На сегодняшнем занятии мы более подробно поговорим про то, почему нейросеть может оказаться более эффективным алгоритмом, чем рассмотренные ранее линейная и логистическая регрессия.

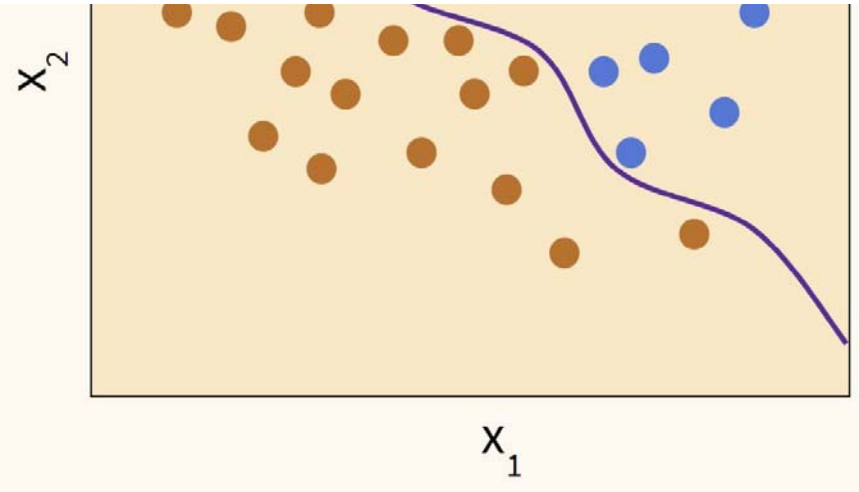
Кроме того, обладая знаниями о производной и градиенте, мы разберем до сих пор неизученный компонент, а именно процесс оптимизации, который в терминологии нейросетей называется обратным распространением, и построим несколько алгоритмов с нуля.

Зачем нужна нейронная сеть

Нелинейная гипотеза

Мы уже знаем, что некоторые гипотезы нелинейны. В случае задачи классификации и алгоритма логистической регрессии это означает, что, как видно на графике ниже, два класса нельзя разделить прямой линией (еще говорят, что они линейно неразделимы).





Можно обучить полиномиальную логистическую регрессию, однако с ростом количества признаков и степени полинома итоговое количество признаков, а значит и «затратность» алгоритма с точки зрения вычислительных ресурсов будет расти.

Для полинома n-ой степени (с одним признаком!) формула выглядит следующим образом.

$$y = \sum_{j=0}^{n} \theta_j x^j$$

Например, полином второй степени будет иметь три коэффициента.

$$y = \theta_0 + \theta_1 x + \theta_2 x^2$$

Полином второй степени *с двумя признаками* уже будет иметь шесть коэффициентов.

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1 x_2 + \theta_5 x_2^2$$

В целом, количество полиномиальных коэффициентов (N) можно рассчитать по формуле.

$$N(n, d) = C(n + d, d),$$

где,

- n — количество линейных признаков;
- d — степень полинома;

- C — количество возможных сочетаний.

Используя пример выше, получим

$$N(2,2) = C(4,2) = 6$$

Полином третьей степени на основе десяти линейных признаков уже потребует создать 286 коэффициентов.

$$N(10,3) = C(13,10) = 286$$

Если речь идет о картинках 28 x 28 пикселей, то после «вытягивания» каждой картинки у нас появится датасет с 784 признаками. Значит, количество членов полинома второй степени составит

$$N(784,2) = C(786,2) = 308505$$

Замечу, что примерное количество признаков полинома второй степени также можно посчитать по формуле $\frac{(n)^2}{2}$, то есть $\frac{(784)^2}{2} = 307328.$

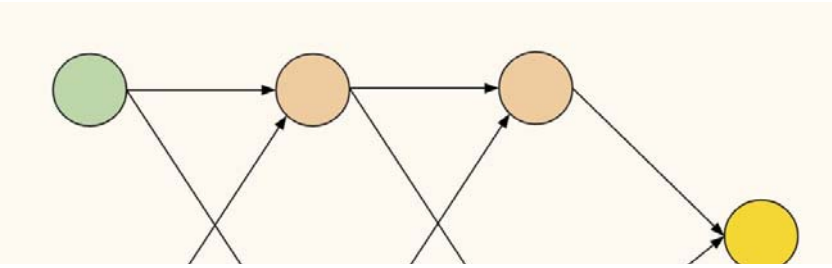
Такое количество признаков потребует очень больших вычислительных ресурсов. Посмотрим, как нейросеть может помочь справиться с этой сложностью.

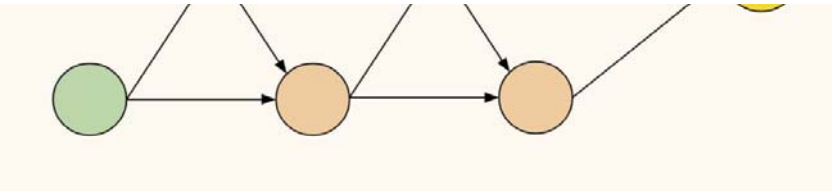
Работа нейронной сети

Рассмотрим работу нейронных сетей с трех различных углов зрения.

Нейрон как дополнительный признак

Возьмем упрощенную модель нейронной сети с двумя скрытыми слоями.



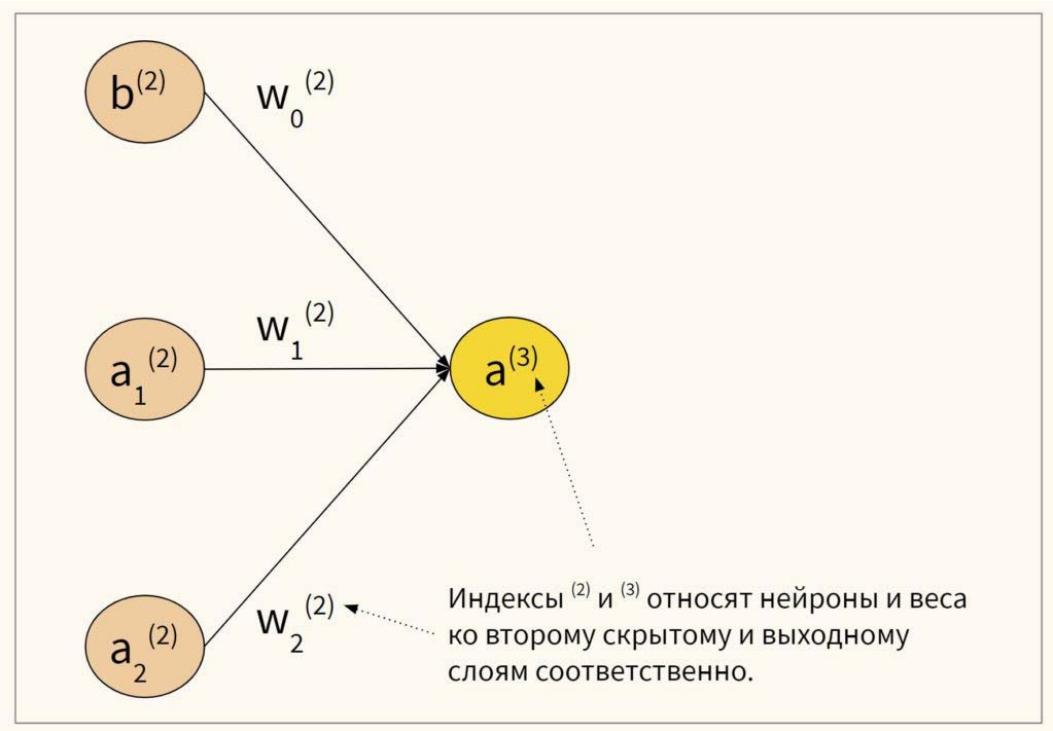


Каждый нейрон каждого из скрытых слоев можно рассматривать как новый, дополнительный признак, зачастую нелинейный, способный уловить то, что не могут уловить линейные признаки. При этом умножение матриц, лежащее в основе работы нейронной сети, и векторизация операций позволяют сделать этот алгоритм достаточно быстрым.

Слой как модель логистической регрессии

Одновременно, если считать, что каждый скрытый слой проходит через функцию активации (activation function), зачастую сигмоиду, то каждый слой, кроме выходного, можно представить как, например, модель логистический регрессии.

Для модели представленной выше рассмотрим, как из второго скрытого слоя получается значение выходного слоя.



В частности,

- на основе первого скрытого слоя мы получили некоторые значения нейронов второго скрытого слоя $a_1^{(2)}$ и $a_2^{(2)}$;
- помимо этого есть вектор весов $w_1^{(2)}$ и $w_2^{(2)}$; и еще
- мы добавим смещение $b^{(2)}$.

Для удобства матричных операций мы можем добавить еще один нейрон скрытого слоя $w_0^{(2)}$ со значением 1 так, как мы это делали, например, в модели линейной регрессии.

Результат умножения двух векторов мы пропустим через сигмоиду g или функцию активации (отсюда выбор буквы a для обозначения этих нейронов) и, таким образом, получим значение выходного слоя $a^{(3)}$. Уверен, вы распознали уравнение логистической регрессии.

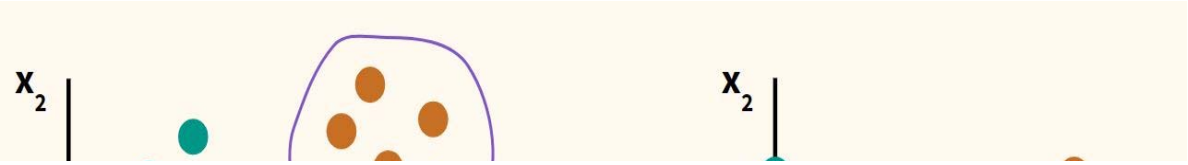
$$a^{(3)} = g(w_0^{(2)} \cdot b^{(2)} + w_1^{(2)} \cdot a_1^{(2)} + w_2^{(2)} \cdot a_2^{(2)})$$

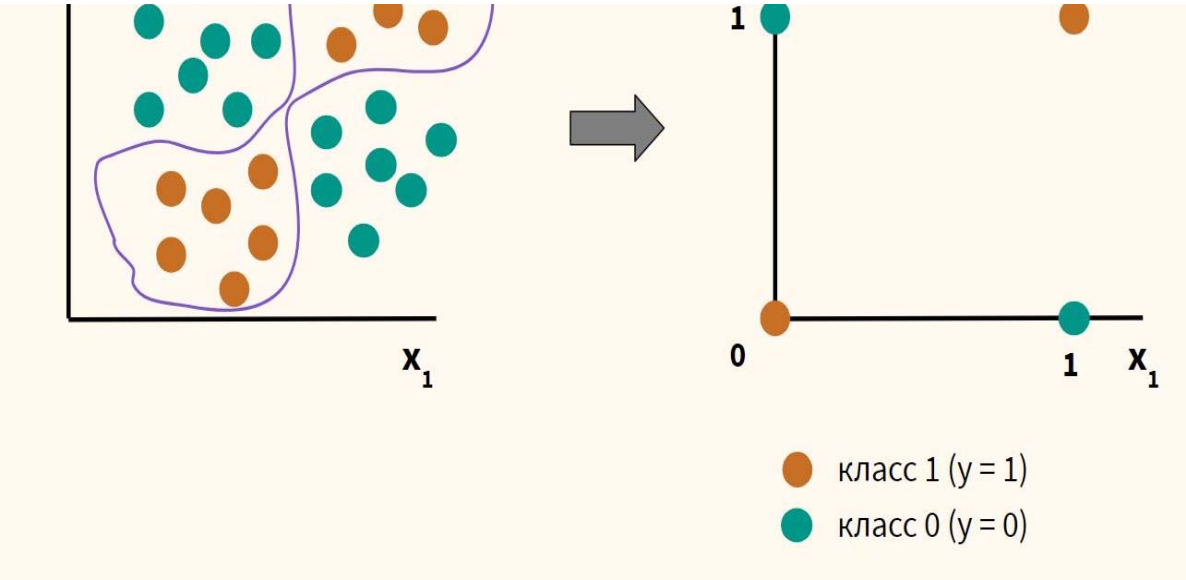
Соответственно, имея два скрытых слоя, мы строим две связанные между собой логистические регрессии. И вдвоем они могут запомнить более сложные зависимости, чем смогла бы одна такая модель, одновременно преодолевая проблему роста количества признаков полиномиальной регрессии.

Нейросеть и таблица истинности

Продемонстрируем, как нейросеть обучается на нелинейной гипотезе с помощью таблиц истинности.

Рассмотрим линейно неразделимые данные двух классов (на рисунке слева) и упростим их до четырех наблюдений, которые могут принимать только значения 0 и 1 (на рисунке справа).





Такая схема соответствует условию **XNOR**. В таблице истинности это условие выглядит так.

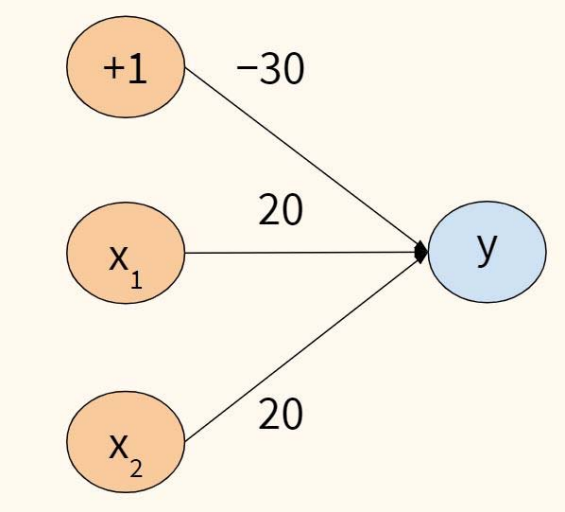
x1	x2	XNOR
0	0	1
0	1	0
1	0	0
1	1	1

Другими словами, когда наблюдение по обоим признакам x_1 и x_2 имеет значение 0 или по обоим признакам имеет значение 1, то результатом будет класс 1. Если же хотя бы один из признаков равен единице, то класс 0.

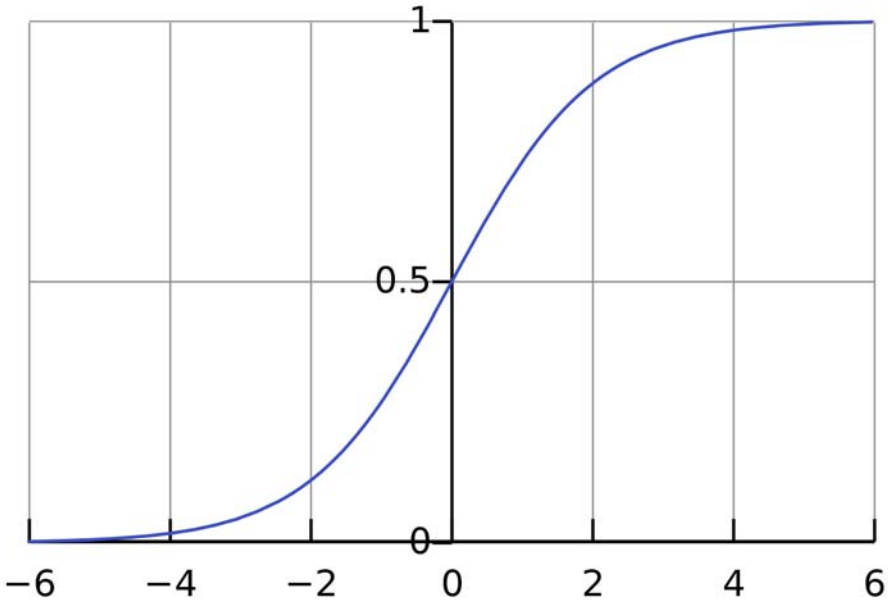
Построим нейросеть, которая будет предсказывать именно такую зависимость. Начнем с более простого компонента, а именно нейросети, которая делает прогноз в соответствии с **логическим И** (AND).

x1	x2	AND
0	0	0
0	1	0
1	0	0
1	1	1

Итак, $x_1, x_2 \in \{0, 1\}$ и $y = x_1 AND x_2$. В нейросети будет два нейрона для признаков и нейрон смещения. Одновременно сразу пропишем веса модели.



Тогда выражение будет иметь вид,
$y_{AND} = sigmoid(-30 + 20x_1 + 20x_2)$. Вспомним, как выглядит график сигмоиды.



Рассмотрим четыре варианта значений x_1, x_2 применительно к такой гипотезе:

- если оба признака будут равны нулю, то результат линейного

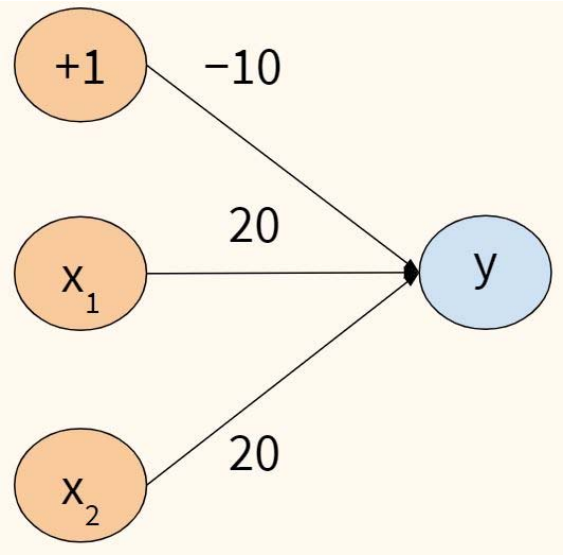
выражения будет равен -30; если «пропустить» это значение через сигмоиду, то результат будет близок к нулю;

- если один из них будет равен нулю, а второй единице, то результат будет равен -10; сигмоида опять выдаст близкое к нулю значение; и только
- если оба признака равны единице, то результат будет равен 10, и сигмоида выдаст значение, близкое к единице.

Это и есть условие логического И. Аналогичным образом можно подобрать веса для **логического ИЛИ** (OR).

x1	x2	OR
0	0	0
0	1	1
1	0	1
1	1	1

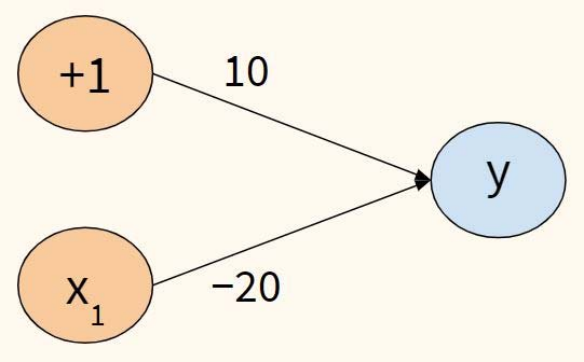
Тогда,



Соответственно $y_{OR} = sigmoid(-10 + 20x_1 + 20x_2)$. Создадим еще более простую сеть для **логического НЕ** (NOT).

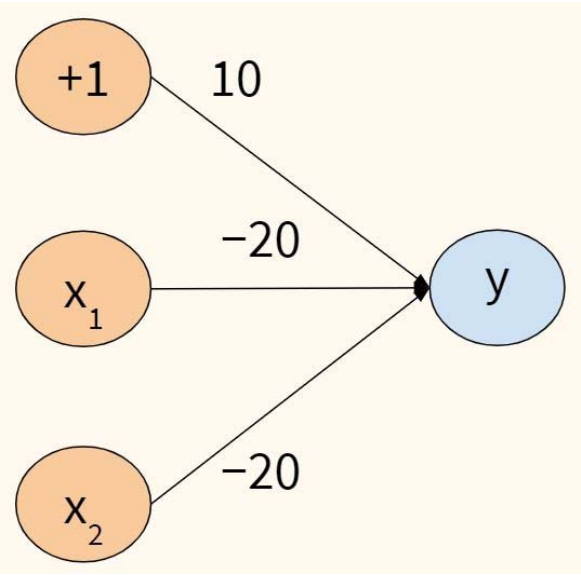
x1	NOT
0	1

1	0

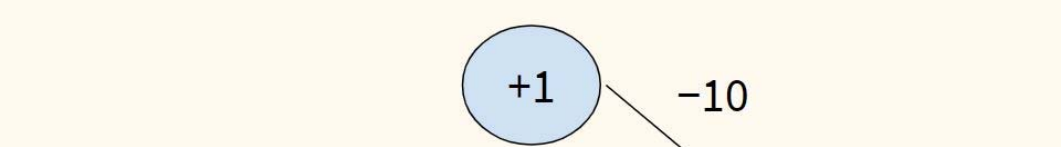


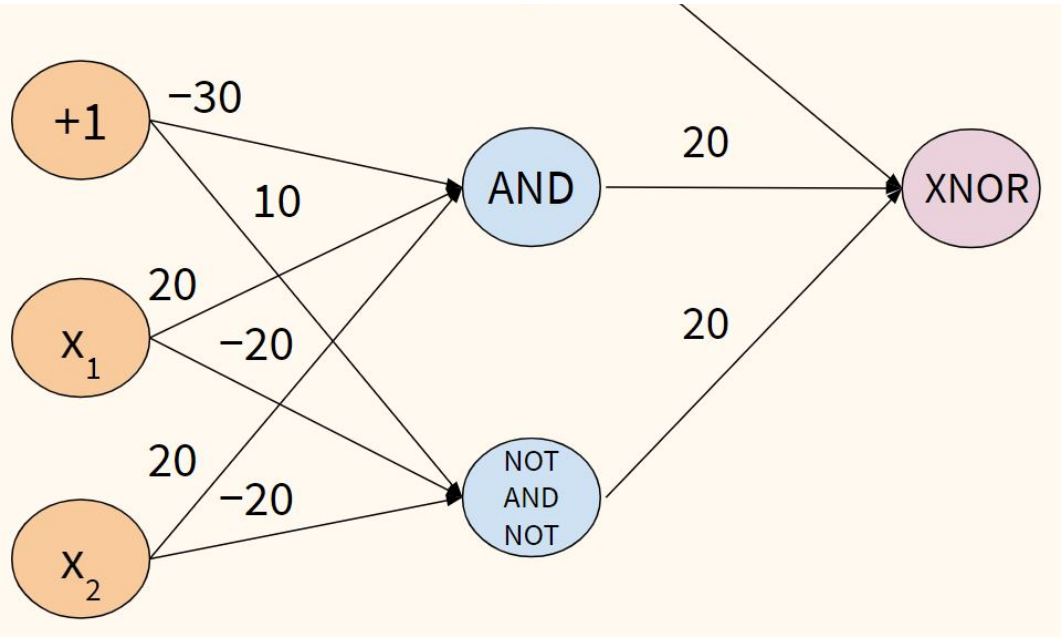
Как следствие, $y_{NOT} = sigmoid(10 - 20x_1)$. Создадим нейросеть, которая будет предсказывать **NOT AND NOT**.

x1	x2	NOT AND NOT
0	0	1
0	1	0
1	0	0
1	1	0



Объединим эти сети в одну.





Рассчитаем таблицу истинности.

x1	x2	AND	NOT AND NOT	XNOR
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

Таким образом, мы видим, что на каждом последующем слое нейросеть строит все более сложную зависимость:

- первый скрытый слой обучился достаточно простым зависимостям AND и NOT AND NOT;
- второй слой дополнил это знание новым OR; и вместе
- они обучились выдавать достаточно сложный результат XNOR.

Подготовка данных

Давайте вновь возьмем данные о вине, построим вначале бинарный, а затем мультиклассовый классификатор и посмотрим, сможет ли нейросеть улучшить показатели логистической регрессии.

Откроем блокнот к этому занятию

Импортируем датасет о вине, удалим класс 2, из признаков оставим спирт и пролин, масштабируем данные.

```
# импортируем датасет
from sklearn import datasets
data = datasets.load_wine()

# сформируем датафрейм и добавим целевую переменную
df = pd.DataFrame(data.data, columns = data.feature_names)
df['target'] = data.target

# удалим класс 2
df = df[df.target != 2]

# оставим только спирт и пролин
X = df[['alcohol', 'proline']]
y = df['target']

# масштабируем признаки
X = (X - X.mean()) / X.std()

# посмотрим на размерность и оставшиеся классы
df.shape, df.target.unique()
```

```
((130, 14), array([0, 1]))
```

Дополнительно преобразуем датафрейм признаков X в массив Numpy с размерность 2 x 130 и сделаем целевую переменную y двумерным массивом.

```
# каждый столбец - это одно наблюдение
X = X.to_numpy().T
X.shape
```

```
(2, 130)
```

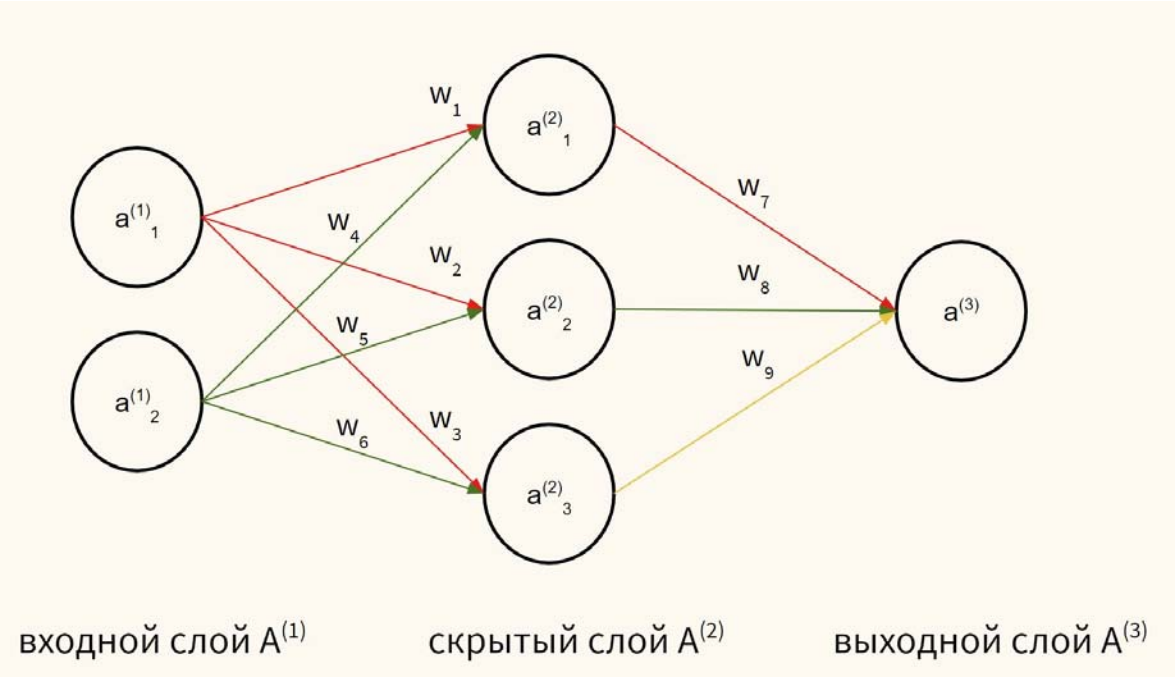
```
y = y.to_numpy().reshape(1, -1)
y.shape
```

```
(1, 130)
```

Нейросеть без смещения

Архитектура сети

Предлагаю идти от простого к сложному и на первом этапе создать алгоритм для бинарной классификации без смещения (bias) с приведенной ниже архитектурой.



Итак, сеть будет состоять из следующих слоев:

- входной слой $A^{(1)}$ из двух нейронов $a_1^{(1)}$ и $a_2^{(1)}$ т.е. двух признаков, на основе которых мы будем предсказывать класс вина;
- один скрытый слой $A^{(2)}$, состоящий из трех нейронов: $a_1^{(2)}$, $a_2^{(2)}$ и $a_3^{(2)}$; и
- выходной слой из одного нейрона $a^{(3)}$, т.е. вероятности принадлежности к одному из двух классов.

Заглавной буквой, например, $A^{(1)}$ обозначаются сразу все нейроны, в данном случае, входного слоя, строчной с соответствющим индексом $a_1^{(1)}$ — отдельный нейрон этого слоя.

Также напомню, что у нас **последовательная** (sequential) архитектура сети, при которой каждый слой получает один тензор на вход и выдает также только один тензор.

Кроме этого, мы используем **полносвязные** (дословно, «плотно связанные», dense, densely connected) **слои**, где каждый нейрон одного слоя связан с каждым нейроном последующего (обратите внимание на красные и зеленые стрелки между входным и скрытым слоями).

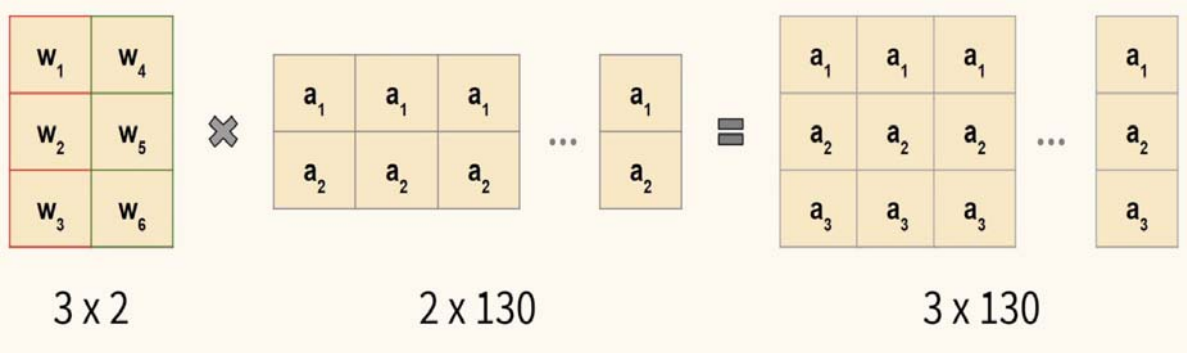
Размерность матриц

Алгоритм нейронной сети, в конечном счете, не более чем умножение матриц или, в общем случае, тензоров (отсюда, в частности, название библиотеки ~~Tensorflow~~, которой мы пользовались, изучая основы нейронных сетей), и после выбора архитектуры модели нам важно определиться с размерностью используемых нами матриц.

Первая матрица весов

Начнем со входного слоя, в который мы одновременно (за счет умножения матриц и векторизации кода) подадим матрицу из 130 наблюдений и двух признаков (после транспонирования размерность, напомню, была 2 x 130).

На нее мы будем умножать матрицу весов $W^{(1)} \cdot X$ или в терминологии слоев $W^{(1)} \cdot A^{(1)}$. Как определить размерность матрицы весов? Очень просто. Нужно взглянуть на количество нейронов скрытого слоя. Их три. Значит размерность первой матрицы весов будет 3 x 2.



$W^{(1)}$ входной слой $A^{(1)}$ скрытый слой $A^{(2)}$

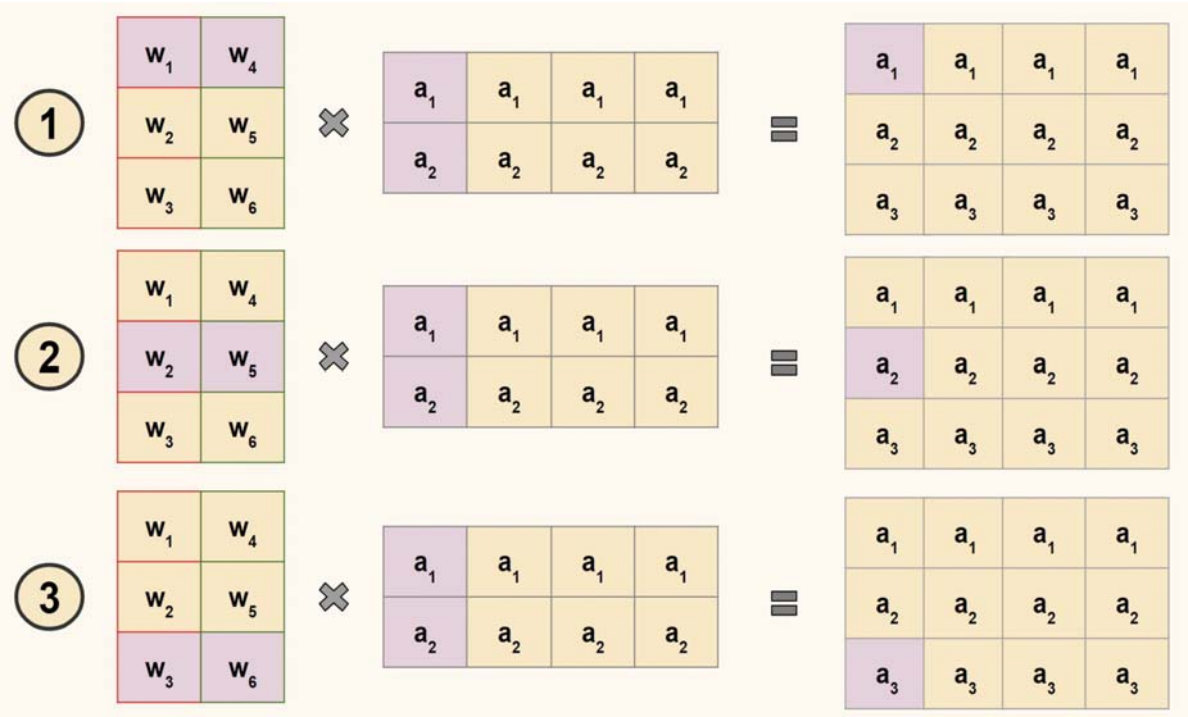
Практический совет. Для того чтобы быстро найти размерность матрицы весов вспомним про две особенности умножения матриц:

- внутренние размеры, т.е. количество столбцов первой и строк второй, должны совпадать, в нашем случае 2 = 2;
- размерность результирующей матрицы будет равна внешним размерам умножаемых матриц, 3 и 130.

Итак, в скрытом слое у нас будет матрица 3 x 130, где каждый столбец — это три (активационных) нейрона для каждого из наблюдений.

Полносвязный слой

Убедимся, что такое умножение матриц обеспечивает умножение каждого нейрона входного слоя на каждый нейрон скрытого слоя (т.е. полносвязность, density, слоев). Для простоты, предположим, что у нас только четыре наблюдения, я не 130.

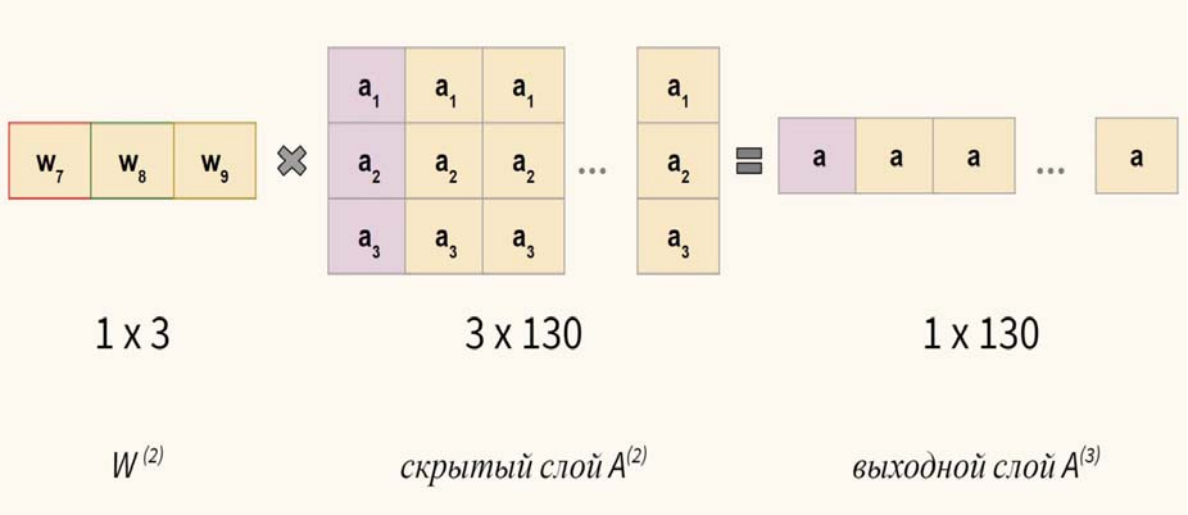


Рассмотрим первую операцию. Здесь веса w_1 и w_4 умножаются на нейроны входного слоя a_1 и a_2 и, таким образом, обеспечивают их «участие» в значении нейрона a_1 скрытого слоя. Аналогично, при второй операции за это отвечают w_2 и w_5. Наконец третий нейрон скрытого слоя рассчитывается благодаря весам w_3 и w_6 и опять же обоим нейронам входного слоя.

Эти же операции можно посмотреть на стрелках на схеме архитектуры сети.

Вторая матрица весов

Теперь, чтобы получить один единственный нейрон выходного слоя (вернее вектор-строку из 130 таких нейронов, 1 x 130), нам нужно новую матрицу весов $W^{(2)}$ умножить на результат скрытого слоя на $A^{(2)}$.



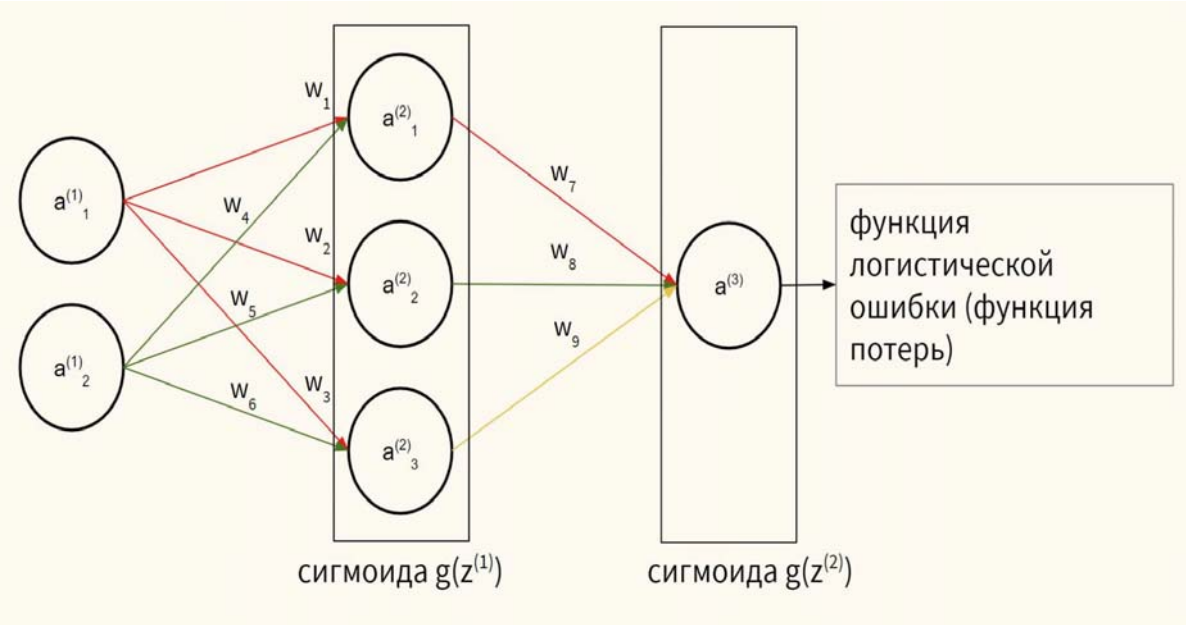
Очевидно, это должна быть матрица 1 x 3, потому что только она даст нам нужную итоговую размерность 1 x 130.

> Важная деталь. Как нейроны скрытого слоя, так и нейрон выходного слоя проходят через **функцию активации** (activation function), в нашем случае сигмоиду (на схемах выше не показана).

В целом мы только что рассмотрели прямое распространение. Давайте напишем соответствующий код.

Код прямого распространения

Вначале объявим знакомые нам функции сигмоиды (функция активации) и логистической ошибки (функция потерь). Их место в архитектуре сети можно увидеть на схеме ниже.



Дополнительно замечу, что в нашей терминологии $z^{(1)}$ и $z^{(2)}$ — это результат умножения матрицы весов на матрицу нейронов соответствующего слоя, который мы «пропускаем» через сигмоиду g. Т.е., например, для скрытого слоя получается

$$Z^{(1)} = W^{(1)} \cdot A^{(1)}$$

$$A^{(2)} = g(Z^{(1)})$$

```
# функция активации
def sigmoid(z):
    s = 1 / (1 + np.exp(-z))
    return s

# функция потерь
def objective(y, y_pred):

  y_one_loss = y * np.log(y_pred + 1e-9)
  y_zero_loss = (1 - y) * np.log(1 - y_pred + 1e-9)
  return -np.mean(y_zero_loss + y_one_loss)
```

Теперь объявим веса и поместим признаки в нейроны скрытого слоя

(исключительно ради единообразия терминологии).

```
# зададим точку отсчета
np.random.seed(33)

# инициализируем случайные веса,
# взятые из стандартного нормального распределения
W1 = np.random.randn(3, 2)
W2 = np.random.randn(1, 3)

# найдем количество наблюдений
n = X.shape[1]

# поместим признаки в нейроны входного слоя
A1 = X
```

Последовательно выполним операции умножения весов на нейроны и пропустим результаты через две сигмоиды.

```
# выполним умножение матриц W1 и A1 и "пропустим" результат через
# в скобках указана итоговая размерность операции
Z1 = np.dot(W1, A1) # (3 x 130)
A2 = sigmoid(Z1) # (3 x 130)

# поступим аналогично с матрицами W2 и A2
Z2 = np.dot(W2, A2) # (1 x 130)
A3 = sigmoid(Z2) # (1 x 130)

# посмотрим, какую вероятность модель выдала для первого наблюдени
A3[:,0]
```

```
array([0.51555295])
```

Найдем ошибку при текущих весах.

```
loss = objective(A3, y)
loss
```

```
7.522291935047792
```

Обратное распространение

Теперь главный вопрос. Как обновить веса так, чтобы уменьшить ошибку?

По большому счету нам нужно рассчитать частную производную функции логистической ошибки L относительно каждого веса $w_1, w_2, w_3, \ldots, w_9$, ведь именно их мы и будем обновлять. Начнем с весов второго слоя, а именно, w_7, w_8, w_9, т.е. $W^{(2)}$.

Частные производные весов $W^{(2)}$

Согласно chain rule градиент (т.е. совокупность частных производных) весов второго слоя будет иметь вид

$$\frac{\partial L}{\partial w^{(2)}} = \frac{\partial L}{\partial a^{(3)}} \odot \frac{\partial a^{(3)}}{\partial z^{(2)}} \odot \frac{\partial z^{(2)}}{\partial w^{(2)}}$$

Нам нужно сделать следующее,

- вначале найти производную функции логистической ошибки $\frac{\partial L}{\partial a^{(3)}}$;
- после этого производную сигмоиды $\frac{\partial a^{(3)}}{\partial z^{(2)}}$; и наконец
- линейной функции $\frac{\partial z^{(2)}}{\partial w^{(2)}}$; затем
- перемножить эти производные.

Возможно вы заметили, что выше использовались индексы (3) и (2), индекс (3) относит активационную функцию $a^{(3)}$ к третьему выходному слою, а линейную функцию $z^{(2)}$ и веса линейной функции $w^{(2)}$ ко второму. В такой нотации нам будет удобнее в дальнейшем рассчитывать градиенты и писать код.

> На всякий случай также уточню, что это будет поэлементное умножение или произведение Адамара (Hadamard product), которое мы будем обозначать через оператор $\odot$.

Далее, уверен, вы обратили внимание на то, что мы выполняем операции в обратном от прямого распространения порядке: сначала производная ошибки, потом сигмоиды третьего слоя, затем линейной функции второго. Именно поэтому процесс называется обратным распространением

ошибки (error back propagation).

Производная функции логистической ошибки

$$\frac{\partial L}{\partial a^{(3)}} = \frac{\partial}{\partial a^{(3)}}\Big(-y\log(a^{(3)}) - (1-y)\log(1-a^{(3)})\Big)$$

Применим правило производной разности и вынесем константы.

$$-y\frac{\partial}{\partial a^{(3)}}\log(a^{(3)}) - (1-y)\frac{\partial}{\partial a^{(3)}}\log(1-a^{(3)})$$

Найдем производную натурального логарифма, вынесем минус за скобку и вычтем одну дробь из другой.

$$-\left(\frac{y}{a^{(3)}} - \frac{(1-y)}{1-a^{(3)}}\right) = \frac{a^{(3)}-y}{a^{(3)}(1-a^{(3)})}$$

Производная сигмоиды

Производную сигмоиды мы уже находили.

$$\frac{\partial a^{(3)}}{\partial z^{(2)}} = g(z^{(2)})(1-g(z^{(2)}))$$

При этом так как результат сигмоиды $g(z^{(2)})$ — это нейрон выходного слоя $a^{(3)}$, то

$$\frac{\partial a^{(3)}}{\partial z^{(2)}} = a^{(3)}(1-a^{(3)})$$

Производная линейной функции

Найдем производную линейной функции, расписав умножение для каждого веса и для каждого нейрона.

$$\frac{\partial}{\partial w^{(2)}}\left(w_7 \times a_1^{(2)} + w_8 \times a_2^{(2)} + w_9 \times a_3^{(2)}\right)$$

Для того чтобы найти производную относительно, например, веса w_7, мы «замораживаем» (считаем константами, производная которых равна нулю) все веса, кроме первого и тогда

$$w_7^{1-1} \times a_1^{(2)} + 0 \times a_2^{(2)} + 0 \times a_3^{(2)} =$$

$$1 \times a_1^{(2)} + 0 \times a_2^{(2)} + 0 \times a_3^{(2)} = a_1^{(2)}$$

Аналогичный результат мы получим, продифференцировав относительно других весов. Тогда,

$$\frac{\partial z^{(2)}}{\partial w^{(2)}} = a^{(2)}$$

Градиент относительно $W^{(2)}$

Наконец перемножим найденные производные и упростим выражение.

$$\frac{\partial L}{\partial w^{(2)}} = \frac{a^{(3)} - y}{a^{(3)}(1 - a^{(3)})} \odot a^{(3)}(1 - a^{(3)}) \odot a^{(2)} =$$

$$(a^{(3)} - y) \odot a^{(2)}$$

В векторной нотации (и матричном умножении) получим

$$\frac{\partial L}{\partial W^{(2)}} = (A^{(3)} - y) \cdot A^{(2)}.T \times \frac{1}{n}$$

Множитель $\frac{1}{n}$ усредняет градиент на количество наблюдений.

Дельта-правило δ_2

Замечу, что $\frac{\partial L}{\partial a^{(3)}} \odot \frac{\partial a^{(3)}}{\partial z^{(2)}}$ также обозначают через греческую букву «дельта» (в нашем случае δ_2), и тогда градиент для обновления весов

$W^{(2)}$, с учетом векторизованного кода, приобретет вид (опять же в векторной нотации)

$$\frac{\partial L}{\partial W^{(2)}} = \delta_2 \cdot A^{(2)}.T \times \frac{1}{n}$$

В дальнейшем использование так называемого «дельта-правила» (delta rule) упростит наш код.

Обновление весов $W^{(2)}$

Остается только обновить веса $W^{(2)}$ в направлении антиградиента, умноженного на коэффициент скорости обучения.

$$W^{(2)} := W^{(2)} - \alpha \times \frac{\partial L}{\partial W^{(2)}}$$

Частные производные весов $W^{(1)}$

Теперь нужно найти производные относительно весов $w_1, \ldots, w_6$ или $W^{(1)}$. И мы снова должны «раскручивать» chain rule от функции логистической ошибки. На этот раз цепь будет более длинной.

$$\frac{\partial L}{\partial w^{(1)}} = \frac{\partial L}{\partial a^{(3)}} \odot \frac{\partial a^{(3)}}{\partial z^{(2)}} \odot \frac{\partial z^{(2)}}{\partial a^{(2)}} \odot \frac{\partial a^{(2)}}{\partial z^{(1)}} \odot \frac{\partial z^{(1)}}{\partial w^{(1)}}$$

Нахождение производных

Вспомним, что **первые два множителя** $\frac{\partial L}{\partial a^{(3)}} \odot \frac{\partial a^{(3)}}{\partial z^{(2)}}$ мы обозначили через δ_2.

Обратим внимание на **третий множитель** $\frac{\partial z^{(2)}}{\partial a^{(2)}}$. В отличие от градиента весов $W^{(2)}$, где мы, напомню, искали производную линейной функции относительно весов $\frac{\partial z^{(2)}}{\partial w^{(2)}}$, здесь нас интересует частная производная относительно нейронов активационного слоя $a^{(2)}$.

Тогда в данном случае мы «замораживаем» (считаем константами) не веса, а нейроны $a^{(2)}$ (считая веса просто числами) и, например, частная производная относительно $a_1^{(2)}$ будет равна

$$\frac{\partial}{\partial a_1^{(2)}}\left(w_7 \times a_1^{(2)} + w_8 \times a_2^{(2)} + w_9 \times a_3^{(2)}\right)$$

$$w_7 \times 1 + w_8 \times 0 + w_9 \times 0 = w_7$$

Аналогично находим производные относительно других нейронов. В векторной нотации,

$$\frac{\partial z^{(2)}}{\partial a^{(2)}} = W^{(2)}$$

Интересно, что ошибкой скрытого слоя E_2 (ошибкой E_1 была бы общая ошибка, которую мы рассчитали с помощью функции логистической ошибки) называют произведение

$$E_2 = \frac{\partial L}{\partial a^{(3)}} \odot \frac{\partial a^{(3)}}{\partial z^{(2)}} \odot \frac{\partial z^{(2)}}{\partial a^{(2)}}$$

Это утверждение более понятно, если переписать (в векторной нотации) выражение выше как

$$E_2 = W^{(2)}.T \cdot \delta_2$$

То есть, мы по сути распространяем «ошибку» δ_2 (число, скаляр) на каждый из трех весов $W^{(2)}$.

Перейдем к **четвертому множителю** $\frac{\partial a^{(2)}}{\partial z^{(1)}}$. Это снова производная сигмоиды, только уже «на слой раньше»,

$$\frac{\partial a^{(2)}}{\partial z^{(1)}} = g(z^{(1)})(1 - g(z^{(1)})) = a^{(2)}(1 - a^{(2)})$$

И наконец **пятый компонент**,

$$\frac{\partial z^{(1)}}{\partial w^{(1)}} = a^{(1)}$$

Дельта-правило δ_1

Аналогично предыдущему слою мы можем обозначить $\frac{\partial L}{\partial a^{(3)}} \odot \frac{\partial a^{(3)}}{\partial z^{(2)}} \odot \frac{\partial z^{(2)}}{\partial a^{(2)}} \odot \frac{\partial a^{(2)}}{\partial z^{(1)}}$ как δ_1 (то есть мы опять взяли все множители, кроме последнего).

Градиент относительно $W^{(1)}$

В итоге градиент относительно весов $W^{(1)}$ имел бы вид,

$$\frac{\partial L}{\partial W^{(1)}} = \left(E_2 \odot A^{(2)} \odot (1 - A^{(2)})\right) \cdot A^{(1)}.T \times \frac{1}{n}$$

Или, раскрыв E_2,

$$\frac{\partial L}{\partial W^{(1)}} = \left((W^{(2)}.T \cdot \delta_2) \odot A^{(2)} \odot (1 - A^{(2)})\right) \cdot A^{(1)}.T \times \frac{1}{n}$$

Или через δ_1

$$\frac{\partial L}{\partial W^{(1)}} = \delta_1 \cdot A^{(1)}.T \times \frac{1}{n}$$

Обратите внимание на паттерн, градиент каждого слоя представляет собой произведение дельты на соответствующие нейроны активационного слоя, усредненное на количество наблюдений.

$$\frac{\partial L}{\partial W^{(2)}} = \delta_2 \cdot A^{(2)}.T \times \frac{1}{n}$$

$$\frac{\partial L}{\partial W^{(1)}} = \delta_1 \cdot A^{(1)}.T \times \frac{1}{n}$$

Разумеется, это правило справедливо и для большего количества скрытых слоев.

Обновление весов $W^{(1)}$

Обновление весов $W^{(1)}$ аналогично предыдущему слою.

$$W^{(1)} := W^{(1)} - \alpha \times \frac{\partial L}{\partial W^{(1)}}$$

Перейдем к коду.

Код обратного распространения

Продолжим писать код, который мы начали создавать, изучая прямое распространение.

```
# найдем дельту весов между слоями 3 и 2
W2_delta = A3 - y # (1 x 130)
```

```
# обратите внимание, это одно число, как и результат
# третьего слоя A3 (мы выводим первый столбец из 130)
W2_delta[:, 0]
```

```
array([0.51555295])
```

```
# найдем дельту весов между слоями 1 и 2
W1_delta = np.dot(W2.T, W2_delta) * A2 * (1 - A2) # (3 x 130)
```

```
# дельта 1 состоит уже из трех чисел, как и скрытый слой нейросети
W1_delta[:, 0]
```

```
array([-0.0099838 ,  0.06300821, -0.01243332])
```

```
# напомню, что умножение дельты 2 на веса скрытого слоя W2 можно
# считать "промежуточной ошибкой" сети или ошибкой скрытого слоя
E2 = np.dot(W2.T, W2_delta)

# то есть одно число W2_delta мы "распространили" на весь скрытый с
# поэтому ошибка состоит из трех чисел
E2[:, 0]
```

```
array([-0.07704936,  1.03666489, -0.04989736])
```

```
# наконец найдем частную производную относительно весов W2
W2_derivative = np.dot(W2_delta, A2.T) / n # (1 x 3)
W2_derivative
```

```
array([[-0.03229078, -0.0351597 ,  0.07797505]])
```

```
# и весов W1
# (размерность опять же должна совпадать с размерностью матриц весо
```

```
W1_derivative = np.dot(W1_delta, A1.T) / n # (3 x 3)
W1_derivative
```

```
array([[-0.00505965, -0.00447514],
       [ 0.05057238,  0.06621354],
       [-0.00678444, -0.00733266]])
```

```
# обновим веса (скорость обучения возьмем равной 0,1)
W2 = W2 - 1 * W2_derivative
W1 = W1 - 1 * W1_derivative
```

Обучение модели

Теперь соединим прямое и обратное распространение и с помощью цикла произведем обучение нейронной сети.

```
np.random.seed(33)

W1 = np.random.randn(3, 2)
W2 = np.random.randn(1, 3)

epochs = 100000
learning_rate = 0.1

n = X.shape[1]

A1 = X

for i in range(epochs):

  # рассчитываем прямое распространение
  Z1 = np.dot(W1, A1) # (3 x 130)
  A2 = sigmoid(Z1) # (3 x 130)

  Z2 = np.dot(W2, A2) # (1 x 130)
  A3 = sigmoid(Z2) # (1 x 130)

  # рассчитываем ошибку
  loss = objective(A3, y)

  # находим дельту весов между слоями 3 и 2
  W2_delta = A3 - y # (1 x 130)
  # находим дельту весов между слоями 2 и 1
  W1_delta = np.dot(W2.T, W2_delta) * A2 * (1 - A2) # (3 x 130)

  # находим частные производные
```

```
  W2_derivative = np.dot(W2_delta, A2.T)/n # (1 x 3)
  W1_derivative = np.dot(W1_delta, A1.T)/n # (3 x 3)

  # обновляем веса
  W2 = W2 - learning_rate * W2_derivative
  W1 = W1 - learning_rate * W1_derivative

  # периодически выводим количество итераций и текущую ошибку
  if i % (epochs / 5) == 0:
    print('Эпоха:', i)
    print('Ошибка:', loss)
    print('-----------------------')
    # можем добавить паузу для более аккуратного вывода
    time.sleep(0.5)

print('Итоговая ошибка', loss)
print('Нейросеть успешно обучена')
```

```
Эпоха: 0
Ошибка: 7.522291935047792
-----------------------
Эпоха: 20000
Ошибка: 1.0093222124524572
-----------------------
Эпоха: 40000
Ошибка: 0.9954032684235387
-----------------------
Эпоха: 60000
Ошибка: 0.9173319445520612
-----------------------
Эпоха: 80000
Ошибка: 0.8744930971601815
-----------------------
Итоговая ошибка 0.845330664541358
Нейросеть успешно обучена
```

Прогноз и оценка качества

Сделаем прогноз и оценим качество.

```
A1 = X

Z1 = np.dot(W1, A1)
A2 = sigmoid(Z1)
Z2 = np.dot(W2, A2)
A3 = sigmoid(Z2)
```

```

# A3.flatten() делает массив одномерным,
# условие выводит True или False (1 или 0)
y_pred, y_true = A3.flatten() >= 0.5, y.flatten()
```

```
W1, W2
```

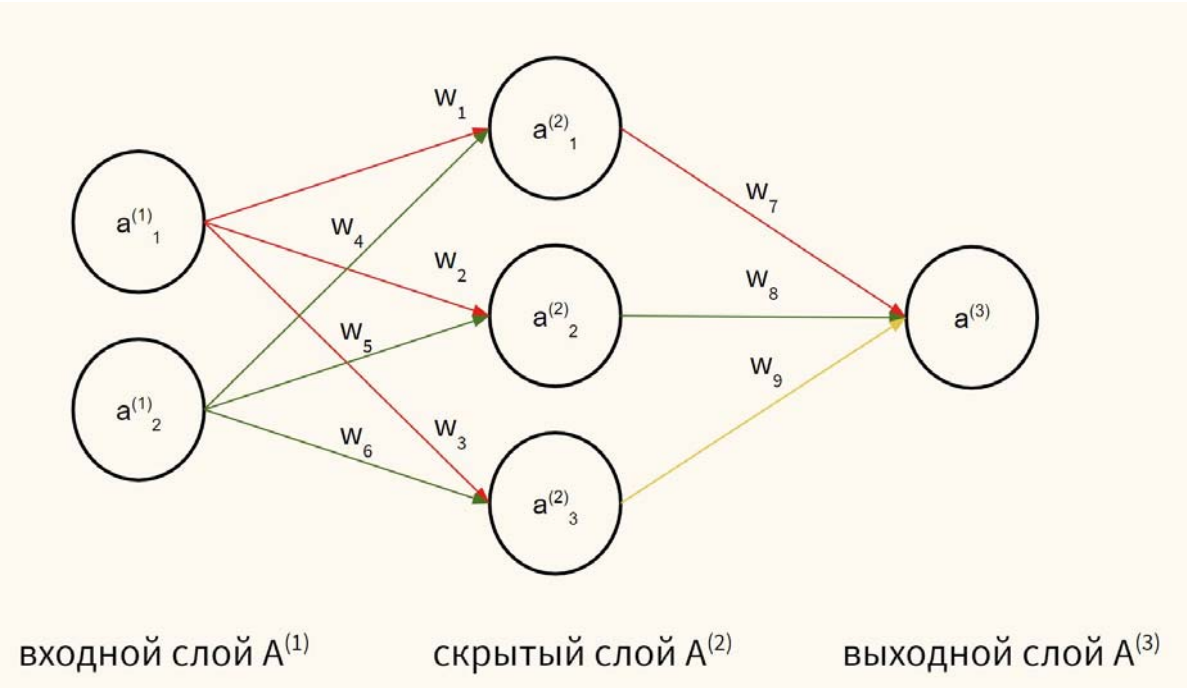
```
(array([[-0.21100871, -1.99383593],
        [-8.00364587,  3.80027268],
        [ 0.18945827,  1.9613264 ]]),
 array([[  6.01782106,   6.42750193, -10.96554005]]))
```

```
accuracy_score(y_true, y_pred)
```

```
0.9769230769230769
```

Инициализация весов

В моделях линейной и логистической регрессии в качестве начальных значений коэффициентов мы использовали нули, в алгоритме нейронной сети — случайные значения, почему так?



Если веса изначально равны нулю, то произойдет несколько нежелательных событий:

- значения активационных слоев $a_1^{(2)} = a_2^{(2)} = a_3^{(2)}$ будут одинаковыми, то есть будут запоминать одну и ту же зависимость;

более того

- поскольку веса между вторым и третьим (выходным) слоями будут одинаковыми, то и значения матрицы δ_2 будут одинаковыми; а значит и
- частные производные, относящиеся к весам одного входного нейрона (например, w_1, w_2, w_3), будут одними и теми же.

Таким образом, после, например, одного обновления весов, хотя значения весов w_1, w_2, w_3 не будут нулевыми, они будут одинаковыми. То же можно сказать про веса w_4, w_5, w_6. И снова $a_1^{(2)} = a_2^{(2)} = a_3^{(2)}$.

Как следствие, мы существенно ограничиваем возможности (гибкость) нейронной сети запоминать сложные зависимости.

Масштабирование целевой переменной

Как мы только что убедились, градиент нейронной сети зависит от целевой переменной. Если эта переменная имеет большой диапазон, то это может создать большую ошибку при вычислении градиента, что, в свою очередь, вызовет существенное изменение весов и дестабилизирует процесс обучения.

Очевидно после обучения и прогноза целевую переменную нужно вернуть к прежнему масштабу.

Модель в Tensorflow и Keras

Библиотека Keras представляет собой надстройку (интерфейс, API), через которую удобно создавать нейросети в библиотеке Tensorflow. Реализуем созданную выше несложную нейросеть в библиотеке Keras.

```
# в Google Colab установлена вторая версия библиотеки Tensorflow,
# которая существенно отличается от первой версии
import tensorflow as tf
tf.__version__
```

```
2.17.0
```

В нейросеть мы будем подавать признаки и целевую переменную таким образом, чтобы объекты были строками.

```
X.T.shape, y.T.shape
```

```
((130, 2), (130, 1))
```

Перейдем к созданию и обучению нейросети.

```
# зададим две точки отсчета, одну в библиотеке Numpy
np.random.seed(42)
# вторую непосредственно в Tensorflow
tf.random.set_seed(42)

# зададим архитектуру: последовательная модель
model = tf.keras.models.Sequential([
  # с полносвязными слоями
  # также укажем нейроны скрытого и выходного слоев
  # откажемся от смещения
  tf.keras.layers.Dense(3, activation = 'sigmoid', use_bias = Fals
  tf.keras.layers.Dense(1, activation = 'sigmoid', use_bias = Fals
])

# зададим особенности стохастического градиентного спуска (SGD)
# в частности, откажемся от импульса
# (подробнее об этом на последующих занятиях)
sgd = tf.keras.optimizers.SGD(learning_rate = 0.1,
                              momentum = 0.,
                              nesterov = False)

# соберем все вместе, дополнительно укажем тип функции потерь и ме
model.compile(optimizer = sgd,
              loss = 'binary_crossentropy',
              metrics = ['accuracy'])

# зададим количество эпох,
# размер batch, после которой мы обновляем веса, равен объему данн
model.fit(X.T, y.T,
          epochs = 10000,
          batch_size = 130,
          verbose = 0)
```

```
<keras.src.callbacks.history.History at 0x7bf76c678a00>
```

```
model.summary()
```

```
Model: "sequential"
```

```
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense (Dense)               (130, 3)                  6

 dense_1 (Dense)             (130, 1)                  3

=================================================================
Total params: 9 (36.00 Byte)
Trainable params: 9 (36.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

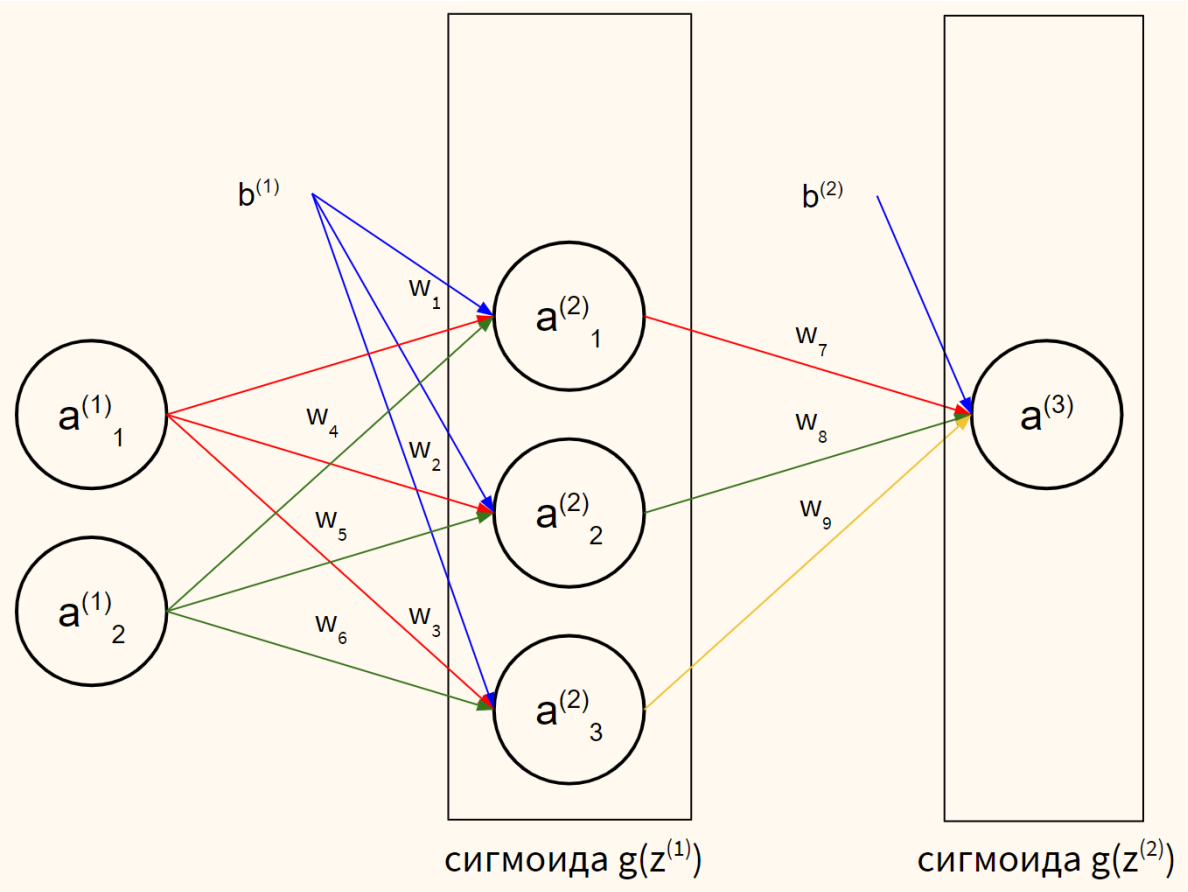
Оценим качество.

```
model.history.history['accuracy'][-1]
```

```
0.9615384340286255
```

Нейросеть со смещением

Добавим смещение, это должно сделать нашу модель более гибкой.



В отличие от линейных моделей, мы не будем использовать одну и ту же производную и для b, и для W (с добавлением столбца из единиц в X). Это связано с тем, что в нейросетях при обратном распространении ошибку на смещение мы *не распространяем*.

Найдем производные смещения относительно сигмоиды (последний компонент в цепи производных).

$$\frac{\partial z^{(2)}}{\partial b^{(2)}} = 1$$

$$\frac{\partial z^{(1)}}{\partial b^{(1)}} = 1$$

Тогда в целом, используя дельта-правило, частные производные относительно $b^{(2)}$ и $b^{(1)}$ будут равны

$$\frac{\partial L}{\partial b^{(2)}} = \sum \delta_2 \times \frac{1}{n}$$

$$\frac{\partial L}{\partial b^{(1)}} = \sum \delta_1 \times \frac{1}{n}$$

Перейдем к коду.

```
np.random.seed(33)

# инициализируем веса
W1 = np.random.randn(3, 2)
# b1 будет иметь размерность 3 x 1,
# потому что распространяется на три нейрона скрытого слоя
b1 = np.random.randn(3, 1)

W2 = np.random.randn(1, 3)
b2 = np.random.randn(1, 1)

n = X.shape[1]

epochs = 100000
learning_rate = 0.1

A1 = X

for i in range(epochs):
```

```
20    # 3 x 2 на 2 x 130
21    Z1 = np.dot(W1, A1) + b1
22    A2 = sigmoid(Z1) # (3 x 130)
23
24    # 1 x 3 на 3 x 130
25    Z2 = np.dot(W2, A2) + b2
26    A3 = sigmoid(Z2) # (1 x 130)
27
28    loss = objective(A3, y)
29
30    W2_delta = A3 - y # (1 x 130)
31    W1_delta = np.dot(W2.T, W2_delta) * A2 * (1 - A2) # (3 x 130)
32
33    # keepdims сохраняет исходную размерность
34    # 1 x 130 на 130 x 3
35    W2_derivative = np.dot(W2_delta, A2.T) / n # (1 x 3)
36    b2_derivative = np.sum(W2_delta, axis = 1, keepdims = True) / n
37
38    # 3 x 130 на 130 x 2
39    W1_derivative = np.dot(W1_delta, A1.T) / n # (3 x 2)
40    b1_derivative = np.sum(W1_delta, axis = 1, keepdims = True) / n
41
42    W2 = W2 - learning_rate * W2_derivative
43    b2 = b2 - learning_rate * b2_derivative
44    W1 = W1 - learning_rate * W1_derivative
45    b1 = b1 - learning_rate * b1_derivative
46
47    if i % (epochs / 5) == 0:
48      print('Эпоха:', i)
49      print('Ошибка:', loss)
50      print('-----------------------')
51      time.sleep(0.5)
52
53  print('Итоговая ошибка', loss)
54  print('Нейросеть успешно обучена')
```

```
1   Эпоха: 0
2   Ошибка: 10.080172158274355
3   -----------------------
4   Эпоха: 20000
5   Ошибка: 0.863497242017787
6   -----------------------
7   Эпоха: 40000
8   Ошибка: 0.6938850880295674
9   -----------------------
10  Эпоха: 60000
11  Ошибка: 0.6439448740818405
12  -----------------------
13  Эпоха: 80000
```

```
Ошибка: 0.6101486239799595
-----------------------
Итоговая ошибка 0.5877500761585371
Нейросеть успешно обучена
```

```
Z1 = np.dot(W1, A1) + b1
A2 = sigmoid(Z1)
Z2 = np.dot(W2, A2) + b2
A3 = sigmoid(Z2)

y_pred, y_true = A3.flatten() > 0.5, y.flatten()

accuracy_score(y_true, y_pred)
```

```
0.9846153846153847
```

```
W1, W2
```

```
(array([[ 2.93497627, -6.10900231],
        [-5.81000119, -4.14576091],
        [ 8.1158836 ,  1.52275952]]),
 array([[ 8.92849304,  7.56210566, -9.22018134]]))
```

TF / Keras. Добавим смещение в нашу модель в библиотеке Keras.

```
np.random.seed(42)
tf.random.set_seed(42)

model = tf.keras.models.Sequential([
  tf.keras.layers.Dense(3, activation = 'sigmoid', use_bias = True
  tf.keras.layers.Dense(1, activation = 'sigmoid', use_bias = True
])

sgd = tf.keras.optimizers.SGD(learning_rate = 0.1,
                              momentum = 0.,
                              nesterov = False)

model.compile(optimizer = sgd,
              loss = 'binary_crossentropy',
              metrics = ['accuracy'])

model.fit(X.T, y.T,
          epochs = 10000,
          batch_size = 130,
          verbose = 0)

model.summary()
```

```
Model: "sequential_5"
_________________________________________________________________
```

```
 Layer (type)                Output Shape              Param #
=================================================================
 dense_10 (Dense)            (130, 3)                  9

 dense_11 (Dense)            (130, 1)                  4

=================================================================
Total params: 13 (52.00 Byte)
Trainable params: 13 (52.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

```
model.history.history['accuracy'][-1]
```

```
0.9615384340286255
```

Два скрытых слоя

Добавим второй скрытый слой.

```
np.random.seed(33)

W1 = np.random.randn(3, 2)
b1 = np.random.randn(3, 1)

W2 = np.random.randn(3, 3)
b2 = np.random.randn(3, 1)

W3 = np.random.randn(1, 3)
b3 = np.random.randn(1, 1)

n = X.shape[1]

epochs = 10000
learning_rate = 0.1

A1 = X

for i in range(epochs):

  Z1 = np.dot(W1, A1) + b1 # (3 x 130)
  A2 = sigmoid(Z1)

  Z2 = np.dot(W2, A2) + b2 # (3 x 130)
  A3 = sigmoid(Z2)

```

```
27    Z3 = np.dot(W3, A3) + b3 # (1 x 130)
28    A4 = sigmoid(Z3)
29
30    loss = objective(A4, y)
31
32    W3_delta = A4 - y # (1 x 130)
33    W2_delta = np.dot(W3.T, W3_delta) * A3 * (1 - A3) # (3 x 130)
34    W1_delta = np.dot(W2.T, W2_delta) * A2 * (1 - A2) # (3 x 130)
35
36    # 3 x 130 на 130 x 3
37    W3_derivative = np.dot(W3_delta, A3.T) / n # (3 x 3)
38    b3_derivative = np.sum(W3_delta, axis = 1, keepdims = True) / n
39
40    # 3 x 130 на 130 x 3
41    W2_derivative = np.dot(W2_delta, A2.T) / n # (3 x 3)
42    b2_derivative = np.sum(W2_delta, axis = 1, keepdims = True) / n
43
44    W1_derivative = np.dot(W1_delta, A1.T) / n # (3 x 2)
45    b1_derivative = np.sum(W1_delta, axis = 1, keepdims = True) / n
46
47    W3 = W3 - learning_rate * W3_derivative
48    b3 = b3 - learning_rate * b3_derivative
49    W2 = W2 - learning_rate * W2_derivative
50    b2 = b2 - learning_rate * b2_derivative
51    W1 = W1 - learning_rate * W1_derivative
52    b1 = b1 - learning_rate * b1_derivative
53
54    if i % (epochs / 5) == 0:
55      print('Эпоха:', i)
56      print('Ошибка:', loss)
57      print('-----------------------')
58      time.sleep(1)
59
60  print('Итоговая ошибка', loss)
61  print('Нейросеть успешно обучена')
```

```
1   Эпоха: 0
2   Ошибка: 11.483275165239569
3   -----------------------
4   Эпоха: 2000
5   Ошибка: 1.2062266445280045
6   -----------------------
7   Эпоха: 4000
8   Ошибка: 1.075310728431138
9   -----------------------
10  Эпоха: 6000
11  Ошибка: 1.0170824508020706
12  -----------------------
13  Эпоха: 8000
```

```
Ошибка: 0.9314962184756368
-----------------------
Итоговая ошибка 0.8359184733738386
Нейросеть успешно обучена
```

```
Z1 = np.dot(W1, A1) + b1
A2 = sigmoid(Z1)
Z2 = np.dot(W2, A2) + b2
A3 = sigmoid(Z2)
Z3 = np.dot(W3, A3) + b3
A4 = sigmoid(Z3)

y_pred, y_true = A4.flatten() > 0.5, y.flatten()

accuracy_score(y_true, y_pred)
```

```
0.9692307692307692
```

Сравним с моделью в **Keras**.

```
np.random.seed(42)
tf.random.set_seed(42)

model = tf.keras.models.Sequential([
  tf.keras.layers.Dense(3, activation = 'sigmoid', use_bias = True
  tf.keras.layers.Dense(3, activation = 'sigmoid', use_bias = True
  tf.keras.layers.Dense(1, activation = 'sigmoid', use_bias = True
])

sgd = tf.keras.optimizers.SGD(learning_rate = 0.1,
                              momentum = 0.,
                              nesterov = False)

model.compile(optimizer = sgd,
              loss = 'binary_crossentropy',
              metrics = ['accuracy'])

model.fit(X.T, y.T,
          epochs = 10000,
          batch_size = 130,
          verbose = 0)

model.summary()
```

```
Model: "sequential_6"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense_12 (Dense)            (130, 3)                  9
```

```

 dense_13 (Dense)            (130, 3)                  12

 dense_14 (Dense)            (130, 1)                  4

=================================================================
Total params: 25 (100.00 Byte)
Trainable params: 25 (100.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

```
model.history.history['accuracy'][-1]
```

```
0.9615384340286255
```

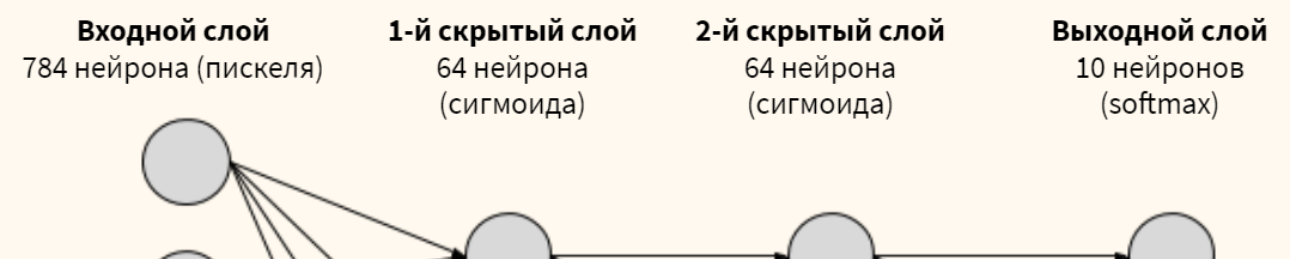
Многоклассовая классификация

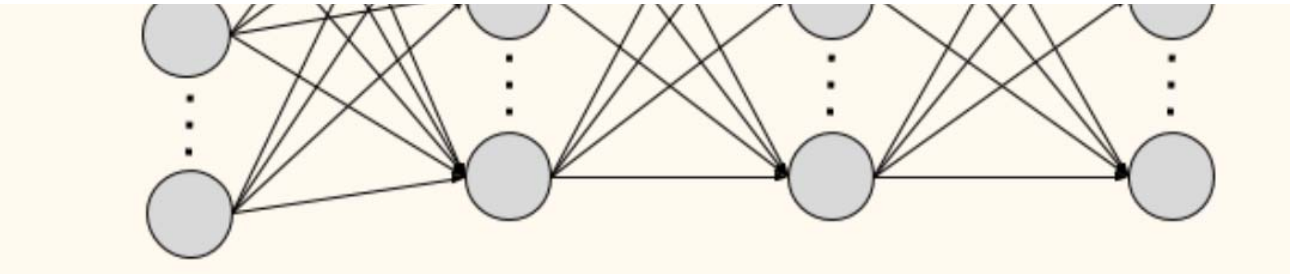
Создадим нейросеть, которая будет предсказывать вероятности нескольких классов. Во многом, идея алгоритма совпадает с softmax логистической регрессией.

Постановка задачи и архитектура

Возьмемся за ту же задачу, которую мы решили в рамках вводного курса с помощью библиотеки Keras, а именно создадим нейросеть, которая будет распознавать рукописные цифры из датасета MNIST.

Архитектуру модели сохраним прежней.





Функции активации

Как видно на графике, мы будем использовать сигмоиду для промежуточных слоев и softmax для выходного слоя.

```
def sigmoid(z):
  s = 1 / (1 + np.exp(-z))
  return s
```

```
def softmax(z):
  # на выходном слое тензор будет иметь размерность 10 x 60000,
  # поэтому складывать мы будем по столбцам
  z = z - np.max(z, axis = 0, keepdims = True)
  numerator = np.exp(z)
  denominator = np.sum(numerator, axis = 0, keepdims = True)
  softmax = numerator / denominator
  return softmax
```

Функция потерь

Функцией потерь будет категориальная кросс-энтропия.

```
def cross_entropy(probs, y_enc, epsilon = 1e-9):
  # опять же, так как softmax выдаст тензор 10 x 60000
  # количество наблюдений содержится в атрибуте shape[1]
  n = probs.shape[1]
  ce = -np.sum(y_enc * np.log(probs + epsilon)) / n

  return ce
```

Обратное распространение

Очевидно, так как изменилась функция потерь и функция активации выходного слоя (softmax), необходимо заново рассчитать производные. Напомню, для весов $W^{(3)}$ цепное правило будет работать следующим образом.

$$\frac{\partial L}{\partial w^{(3)}} = \frac{\partial L}{\partial a^{(4)}} \odot \frac{\partial a^{(4)}}{\partial z^{(3)}} \odot \frac{\partial z^{(3)}}{\partial w^{(3)}}$$

При этом, оказывается, что производная первых двух компонентов $\frac{\partial L}{\partial a^{(4)}} \odot \frac{\partial a^{(4)}}{\partial z^{(3)}}$ (т.е. кросс-энтропии и softmax) сводится к $a^{(4)} - y$ (она аналогична бинарной кросс-энтропии и сигмоиде, но находится, разумеется, иначе).

Одновременно этот компонент производной представляет собой δ_3, которую для нахождения градиента необходимо умножить на $A^{(3)}.T$.

$$\frac{\partial L}{\partial W^{(3)}} = \delta_3 \cdot A^{(3)}.T \times \frac{1}{n}$$

Остальные производные находятся аналогично предыдущим моделям.

$$\frac{\partial L}{\partial W^{(2)}} = \delta_2 \cdot A^{(2)}.T \times \frac{1}{n}$$

$$\frac{\partial L}{\partial W^{(1)}} = \delta_1 \cdot A^{(1)}.T \times \frac{1}{n}$$

Подготовка данных

```
!pip install mnist
```

```
import mnist
from tensorflow import keras
```

```
def ohe(y):
  examples, features = y.shape[0], len(np.unique(y))
  zeros_matrix = np.zeros((examples, features))
  for i, (row, digit) in enumerate(zip(zeros_matrix, y)):
    zeros_matrix[i][digit] = 1

  return zeros_matrix
```

```
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_da
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
X_train = 2. * (X_train - np.min(X_train)) / np.ptp(X_train) - 1
X_test = 2. * (X_test - np.min(X_test)) / np.ptp(X_test) - 1

X_train = X_train.reshape((-1, 784)).T
X_test = X_test.reshape((-1, 784)).T

y_train_enc, y_test_enc = ohe(y_train).T, ohe(y_test).T

X_train.shape, y_train_enc.shape
```

```
((784, 60000), (10, 60000))
```

Обучение модели

```
np.random.seed(33)

W1 = np.random.randn(64, 784)
b1 = np.random.randn(64, 1)

W2 = np.random.randn(64, 64)
b2 = np.random.randn(64, 1)

W3 = np.random.randn(10, 64)
b3 = np.random.randn(10, 1)

n = X_train.shape[1]

epochs = 2000
learning_rate = 0.1
loss_history = []

A1 = X_train

for i in range(epochs):

  # 64 x 784 на 784 x 60000 --> 64 x 60000
  Z1 = np.dot(W1, A1) + b1
  A2 = sigmoid(Z1)

  # 64 x 64 на 64 x 60000 --> 64 x 60000
  Z2 = np.dot(W2, A2) + b2
  A3 = sigmoid(Z2)
```

```python

  # 10 x 64 на 64 x 60000 --> 10 x 60000
  Z3 = np.dot(W3, A3) + b3
  A4 = softmax(Z3)

  loss = cross_entropy(A4, y_train_enc)

  W3_delta = A4 - y_train_enc # (10 x 60000)
  W2_delta = np.dot(W3.T, W3_delta) * A3 * (1 - A3) # (64 x 60000)
  W1_delta = np.dot(W2.T, W2_delta) * A2 * (1 - A2) # (64 x 60000)

  # 10 x 60000 на 60000 x 64 --> 10 x 64
  W3_derivative = np.dot(W3_delta, A3.T) / n
  b3_derivative = np.sum(W3_delta, axis = 1, keepdims = True) / n

  # 64 x 60000 на 60000 x 64 --> 64 x 64
  W2_derivative = np.dot(W2_delta, A2.T) / n
  b2_derivative = np.sum(W2_delta, axis = 1, keepdims = True) / n

  # 64 x 60000 на 60000 x 784 --> 64 x 784
  W1_derivative = np.dot(W1_delta, A1.T) / n
  b1_derivative = np.sum(W1_delta, axis = 1, keepdims = True) / n

  W3 = W3 - learning_rate * W3_derivative
  b3 = b3 - learning_rate * b3_derivative
  W2 = W2 - learning_rate * W2_derivative
  b2 = b2 - learning_rate * b2_derivative
  W1 = W1 - learning_rate * W1_derivative
  b1 = b1 - learning_rate * b1_derivative

  loss_history.append(loss)

  if i % (epochs / 5) == 0:
    print('Эпоха:', i)
    print('Ошибка:', loss)
    print('-----------------------')

print('Итоговая ошибка', loss)
print('Нейросеть успешно обучена')
```

```
Эпоха: 0
Ошибка: 6.215969648082527
-----------------------
Эпоха: 400
Ошибка: 1.3167396354155767
-----------------------
Эпоха: 800
Ошибка: 1.0250926474543347
-----------------------
```

```
Эпоха: 1200
Ошибка: 0.8745745950320921
-----------------------
Эпоха: 1600
Ошибка: 0.7776535165416646
-----------------------
Итоговая ошибка 0.7097715571543584
Нейросеть успешно обучена
```

Прогноз и оценка качества

Сделаем прогноз и оценим качество на обучающей выборке.

```
Z1 = np.dot(W1, A1) + b1
A2 = sigmoid(Z1)
Z2 = np.dot(W2, A2) + b2
A3 = sigmoid(Z2)
Z3 = np.dot(W3, A3) + b3
A4 = softmax(Z3)
```

```
y_pred = np.argmax(A4, axis = 0)
y_pred[:4]
```

```
array([5, 0, 9, 1])
```

```
accuracy_score(y_train, y_pred)
```

```
0.7677833333333334
```

Теперь на тестовых данных.

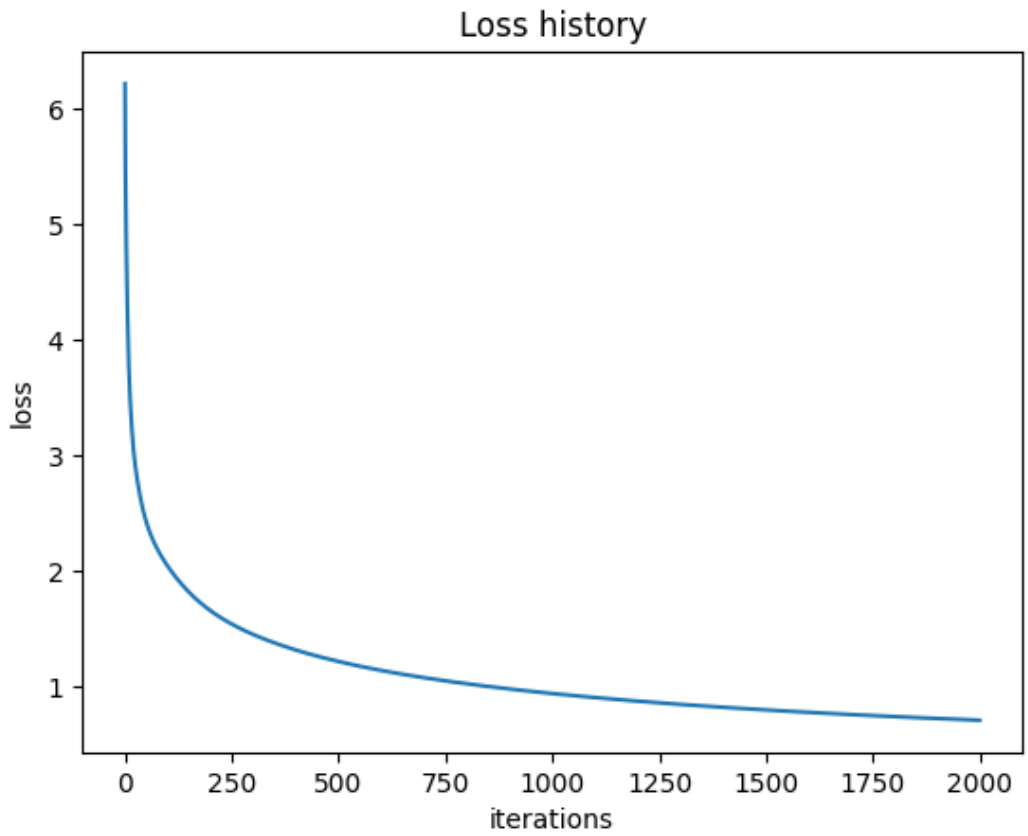
```
A1 = X_test

Z1 = np.dot(W1, A1) + b1
A2 = sigmoid(Z1)
Z2 = np.dot(W2, A2) + b2
A3 = sigmoid(Z2)
Z3 = np.dot(W3, A3) + b3
A4 = softmax(Z3)

y_pred = np.argmax(A4, axis = 0)

accuracy_score(y_test, y_pred)
```

```
0.7658
```

Дополнительно оценим изменение уровня ошибки. Когда мы используем

все имеющиеся данные для расчета градиента, «скачков» на графике быть не должно, ошибка должна постоянно уменьшаться.

```
plt.plot(loss_history)
plt.title('Loss history')
plt.xlabel('iterations')
plt.ylabel('loss');
```



При увеличении количества эпох и настройке скорости обучения алгоритм мог бы показать более высокий результат. При этом очевидно, что модель, которую мы создали в рамках вводного курса в библиотеке Keras, обучилась гораздо быстрее. На занятии по стохастическому градиентному спуску мы посмотрим, как можно ускорить работу нашего алгоритма.

Подведем итог

На сегодняшнем занятии мы в деталях рассмотрели математику обратного распространения, а также с нуля построили несколько алгоритмов нейронных сетей.

Ответы на вопросы

Вопрос. Чем отличается умножение матриц от векторизации?

Ответ. Принцип умножения матриц относится к математике и описывает правила, по которым мы умножаем два двумерных тензора.
Векторизация кода — термин из программирования, который позволяет избежать использования циклов в процессе выполнения кода.